Visual Odometry in Monocular Vision: Study and Analysis

Xiaowei CHEN & Bo WANG

***Abstract*— Visual odometry is a relatively new discipline and technology. We will study it from zero and compare how it performs in different situations.**

I. Introduction

The objective is to study the application of the Visual odometry algorithm. We take a photo sequence as a data set with a monocular camera. Knowing that the monocular vision does not allow to distunguate the depth, but it compensates its deficit by cognitive cues. For example: 1)Parallel lines meet at the horizon. 2)The further away the objects are, the smaller they are. 3)The movements of objects differ according to the distance at which they are located. For this, a sequence of photos taken from different angles is necessary. We use Python as our tool to study. We use the reference code on the link https://github.com/uoip/monoVO-python, and modify it to fit our project.

II. Calibration

A. Introduction

The accuracy of the camera is very important for the image recognition program. When we calibrate a camera, it is essentially to determine its intrinsic parameters that do not change if we move the camera. The camera matrix is in the form :

$$camera\ matrix = \begin{pmatrix} fx & 0 & cx \\ 0 & fy & cy \\ 0 & 0 & 1 \end{pmatrix}$$

B. Use of the program

OpenCV contains a calibration function that automatically gives the required camera matrix with the successful distinguished images. And this camera matrix can be used on all photographic actions with the same camera. So before using VO, you need to run the calibration program *calibration.py*. **Before using this program, you need to change the path of the image (line 29).**

```
29    images = glob.glob('your path' + '/*.JPG')
```

Figure 2.Line need customization in calibration.py

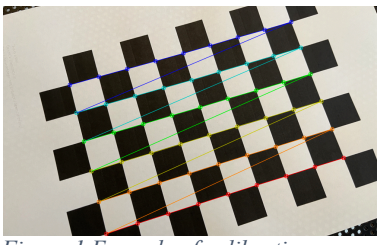


Figure 1.Exemple of calibration

We used the camera of the iphone 13 mini to take pictures, and the resolution of the photos was 4032*3024. Because the resolution of the camera is too high and the calculation takes long time (probably several hours), we use the *cv2.resize* function to change the image to 0.25 times its original size. This will affect the intrinsic parameter we get. So later when calculating VO, we have to do the same operation. Here we give the calibration results directly: fx=816 , fy=818, cx=506, cy=344.

III. Implementation

Everytime before running the main function, you need to pay attention to：

Line 7: the arguments of function *PinholeCamera*() is : picture's width, picture's height, fx, fy, cx,cy

```
7    cam = PinholeCamera(4032, 3024, 816, 818, 506, 344)
```

Figure 3. Line need customization in main.py

Line 12 : the range of loop is the ID number of image

Line 13 : your path of folder

```
12   for img_id in range(4035,4080):
13       img = cv2.imread("your path/pic/IMG_"+str(img_id)+".JPG", 1)
```

Figure 4. Line need customization in main.py

For exemple, this figure means you want to load image from "IMG_4035.JPG" to "IMG_4080.JPG".

After launching main.py, a window with an image will appear. At this point you may need to close it manually for him to continue running.

We use the LK optical flow method, *cv2.calcOpticalFlowPyrLK(),* which describes the instantaneous velocity of a pixel in the two following images based on the correlation between the consecutive images in the time domain. We first read the first image, Then we convert it into grayscale which will be prescribed as input to the algorithm.

For the first image, we first perform the detection of the key points, here we use the Fast[1] algorithm, later we will compare with other different algorithms. Then we record the results for the second image. By combining the first and second images and the recorded keypoints, we can track the movement of the keypoints in the image. The opencv API automatically computes the key points in the second image and generates a mask, which is also used to filter out some of the redundant points.

IV. ANALYSE

To compare the differences between the 3 algorithms, we will run the same photo sequence 3 times with different algorithms. In order to better analyse the trajectory, we will first take a straight path with fixed camera pose.

A. Detection of key points

We put the desired images in a folder with the name 'pic', which is already created in the rendered zip. To perform the detection with different algorithms, we have to comment and/or uncomment lines of code in the file 'visual_odometry.py':

- Uncomment these lines for the Feature to Track algorithm, we need to uncomment lines 68-72 and comment on lines 67 and 73, and put them as shown below:

```
65    def processFirstFrame(self):
66        start = time.perf_counter()
67        #self.px_ref = self.detector.detect(self.new_frame)
68        self.px_ref = cv2.goodFeaturesToTrack(self.new_frame,5000,0.01,10)
69        local_px_ref = np.int0(self.px_ref)
70        for i in local_px_ref:
71            x,y = i.ravel()
72            cv2.circle(self.img_rgb,(x,y),3,255,-1)
73        #img = cv2.drawKeypoints(self.img_rgb, self.px_ref, self.img_rgb)
74        cv2.imshow("img",self.img_rgb)
75        cv2.waitKey(0)
76        cv2.imwrite('results/lol' + str(self.counter) + '.png', img)
77        self.counter+=1
```

Figure 5. Line need comment and uncomment in visual_odometry.py

It gives us:



Figure 6. Detection of key points

This is a simple method that detects crosses of lines, this method is not very suitable for our project, because these points move irregularly with the camera changing its position.

- We can choose one of algorithm FAST, SIFT and ORB by commenting the two lines of 51,52,53 in *visual_odometry.py*, and the lines in second figure should be commented.

```
51        self.detector = cv2.FastFeatureDetector_create(threshold=25, nonmaxSuppression=True)
52        #self.detector = cv2.SIFT_create()
53        #self.detector = cv2.ORB_create(500, nlevels=15)
```

```
68        #self.px_ref = cv2.goodFeaturesToTrack(self.new_frame,5000,0.01,10)
69        #local_px_ref = np.int0(self.px_ref)
70        #for i in local_px_ref:
71        #    x,y = i.ravel()
72        #    cv2.circle(self.img_rgb,(x,y),3,255,-1)
```

Figure 7. FAST chosen for exemple

This allows us to plot the point detection image in the 'results' folder (this folder is also created in the rendering)

We first take a sequence of pharmacy pictures and process them with the FAST algorithm, for this you can put the pictures in the 'pharmacie' folder into the 'pic' folder, then change the name(number) and condition in the *main.py*, according to the following figure:(same for the SIFT and ORB algorithm)

```
12  for img_id in range(4035,4073):
13      img = cv2.imread("pic\\IMG_"+str(img_id)+".JPG", 1)
```

We change the numbers in line 12 and 13 to match the names of the images in 'pic'. In the rest of this work, we have to change these numbers and the images in 'pic' several times with the same method, so we will ignore this step in the rest of the report.

We get directly: (the figures are created automatically in the folder 'results' after the program is launched)



Figure 8. Method FAST



Figure 9. Method SIFT



Figure 10. Method ORB

From the previous images, we can deduce that the FAST algorithm detects the most key points and the ORB algorithm the least. We will do the analysis with its trajectory in the following report

We also have a 'magasin' folder which is a photo sequence with fewer key points detected to check our result. (In the pharmacy sequence, the trees have too many key points, but it doesn't make much effort.

B. *Trajectory*

Our program gives directly the trajectory of the detected key points for these algorithms in the same file with codes, with the image named 'map.png'. As we have indicated that we take the sequence of pictures with the camera moving along a straight line, we can better analyse the trajectories, according to this we are able to determine the movement of the camera.

The coordinates are not really exact because lack of real scale, so the values of x, y and z are not logic, but their proportion seems correct. And what we are interested in is the orientation and shape of the trajectory.

The green point is the starting point. The white points is the step.

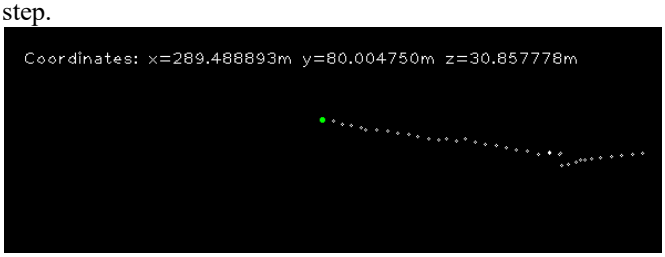


Figure 11.Trajectoire FAST

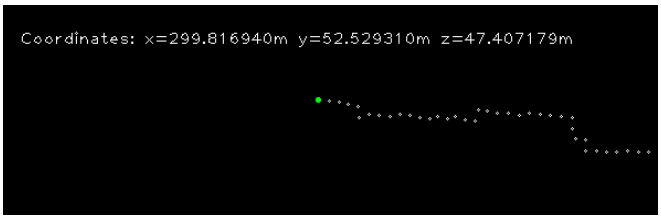


Figure 12.Trajectoire SIFT

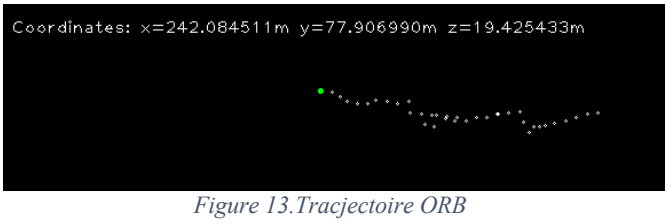


Figure 13.Tracjectoire ORB

These figures are directly created in the main folder with the name 'map.png'. By taking into account the detection images of the previous key points, we can deduce that the more points we detect, the more delicate the trajectory is, for the three algorithms, we see that at the end of the operation, the trajectory is not adapted, this is because the photos are not perfect, there are some crossers in the street. And maybe we didn't keep the line straight by taking pictures at the end (knowing that we already deleted some pictures at the baganing and the end of the operation).

As we have made these images keeping a straight line, we can simply prove it according to the previous trajectory figures.

Unfortunately, we can't plot the optic flow pattern, we tried to make drawings for e.g. key points of 2 images on one, but we only got confusing images.

C. *Customized quantitative*

In this part, we will add additional variables to our test. With the previous results, we can say that the FAST method is more robust, so we will use this algorithm for the rest of the projet.

- Light on/off

  We study the effect of light on the algorithm. To simplify this step, we print a sheet of paper with a chinese character, and take a series of pictures by moving around the parper with the light on and off.

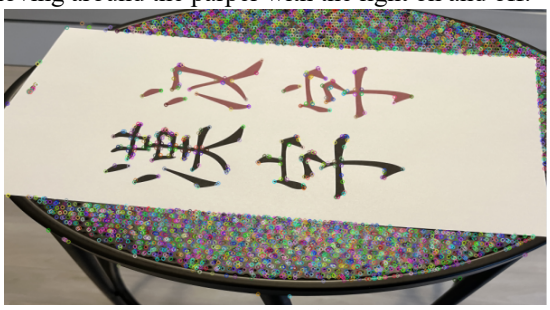


Figure 14.Key points when light on

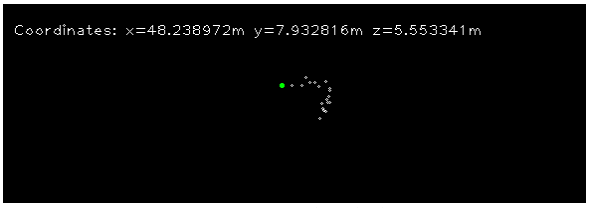


Figure 15.Trajectory when light on

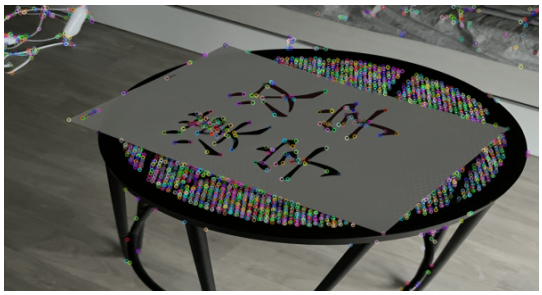


Figure 16. Key points when light off

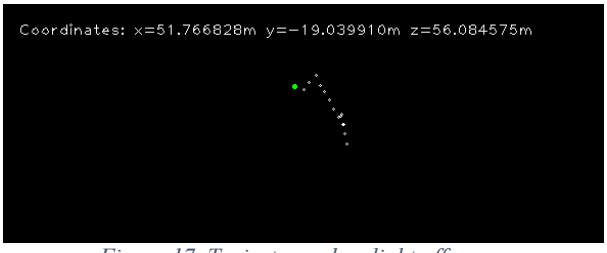


Figure 17. Trajectory when light off

We can find that the light bring some differences, but the general trend is the same. Since there is no guarantee that we take the exact same route when we take photo, we can assume that this algorithm is not very sensitive to light.

- Mouvement of camera fast/slow

  We study the impact of the speed of camera movement. To simulate fast movement, we can simply skip frames during sampling photo. It means add a step(step=3 for example) for image processing.

```
12  for img_id in range(4035,4074,3):
```

Figure 18. To simulate 3 times speed, we add a step=3 when sampling photo in main.py

We show here an exemple of 'pharmacy':

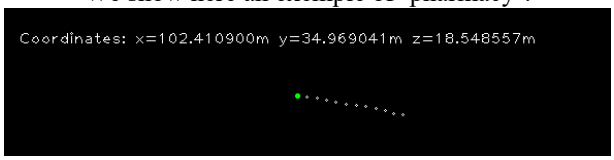


Figure 19. Trajectoire of 'pharmacy' with high speed

According to figure(11) and figure(19), we can see that the shape of the trajectory does not change, but the movement distance has become shorter. We can assume that the movement speed has a effect on the distance estimate.

- Mouvement of the camera in translation/rotation

  We take two sequences of photos with road singling, one with translational camera movement, and one with rotation. They are in file 'singalisation_rotation' and 'signalisation translation'. In this section of translation, we go straight to the road singling because we have did the part passe by the scène before with the pharmacy.

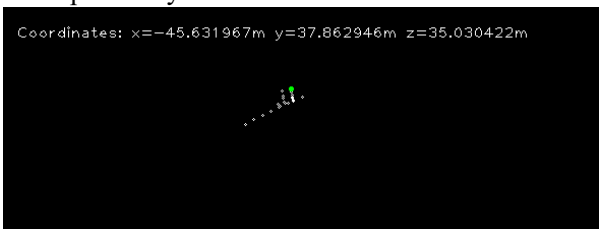


Figure 20. Trajectoire translation

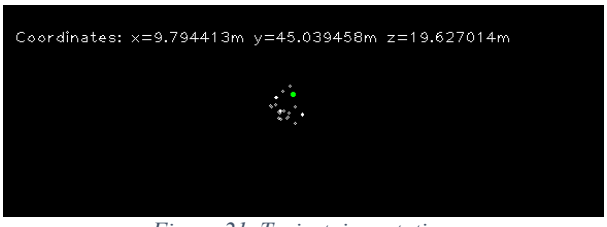


Figure 21. Trajectoire rotation

We can see that for the camera in translation straight to the singling, the points of the trajectory remain in a zone towards the middle, because theoretically the camera spotlight is supposed to be fixed in a 2D.

Then we put the camera in rotation around the road sign, we can deduce that the same trajectory as in the previous translation, item the centre of the camera field is 'fixed' in the world field.

- Text comparison
  We place a piece of paper with the letter 'X' on the same bench, then take a picture of it in the same light condition as before by doing straight line movements and calculate it. The result is as follows :



Figure 22. Key points of 'X'

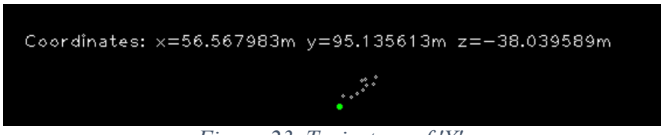


Figure 23. Trajectory of 'X'

The trajectory detected is correct. This could indicate that the problem of different languages does not have a significant impact on this algorithm.

V. Conclusion

A conclusion section is not required. Although a conclusion may review the main points of the paper, do not replicate the abstract as the conclusion. A conclusion might elaborate on the importance of the work or suggest applications and extensions.

Appendix

A *dataset.zip* which contient all the photo we used.

References

[1] https://github.com/uoip/monoVO-python